Bachelor of Computer Applications (Hons. AI and ML)
Semester- III

Subject: Database Management System

Assignment 3: Database Design Analysis

By
Name: AKASH P
Reg. No.: 2511021240008

Faculty Name: Renita Joy
Faculty Signature:_____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
October 2026

1. Introduction

This analysis continues the Online Food Delivery System designed in Assignments 1 and 2. The system uses 11 related tables to manage customers, delivery addresses, restaurant owners, restaurants, menu categories, menu items, delivery agents, orders, order items, payments, and reviews. The purpose of separating these entities is to reduce repeated facts and keep the data consistent while an order moves from placement to payment, delivery agent assignment, and delivery.

2. Data redundancy and anomalies

Data redundancy occurs when the same fact is stored repeatedly. For example, if a single table stored every order line together with the customer, address, restaurant, owner, menu item, delivery agent, and payment, then the customer's name and phone, the restaurant's details, and the agent's vehicle number would be repeated on every row of every order. The submitted design avoids much of this repetition by storing those facts once in separate tables and connecting them through keys.

Such repetition causes three kinds of anomalies in a single-table design:

- **Update anomaly:** if a restaurant changes its phone number or a menu item's price, every repeated row would have to be changed, and missing one leaves inconsistent data.
- **Insertion anomaly:** a new restaurant, menu item, or delivery agent could not be stored until it appears in an order.
- **Deletion anomaly:** deleting the only order of a customer or restaurant would also delete the customer's or restaurant's details.

In the proposed schema, each of these facts lives in exactly one row of its own table, so all three anomalies are avoided for the main entities.

3. Functional dependencies

A functional dependency X → Y means that a value of attribute set X determines exactly one value of attribute set Y. The following dependencies are supported by the proposed schema. Each primary key determines the non-key attributes in its own table.

Relation	Main functional dependencies
Customers	customer_id → name, email, phone, registration_date; email → customer_id (UNIQUE)
Addresses	address_id → customer_id, address_line, city, pincode, address_type
Restaurant_Owners	owner_id → name, email, phone; email → owner_id (UNIQUE)
Restaurants	restaurant_id → owner_id, name, phone, address, cuisine_type, opening_status
Categories	category_id → category_name, description; category_name → category_id (UNIQUE)
Menu_Items	item_id → restaurant_id, category_id, name, description, price, is_veg, availability; (restaurant_id, name) → item_id (UNIQUE)
Delivery_Agents	agent_id → name, phone, vehicle_number, availability_status; phone → agent_id and vehicle_number → agent_id (UNIQUE)
Orders	order_id → customer_id, restaurant_id, address_id, agent_id, order_datetime, order_status, total_amount
Order_Items	(order_id, item_id) → quantity, unit_price

Relation	Main functional dependencies
Payments	payment_id → order_id, amount, payment_method, payment_status, transaction_ref, payment_datetime; order_id → payment_id and transaction_ref → payment_id (UNIQUE)
Reviews	review_id → order_id, rating, comment, review_date; order_id → review_id (UNIQUE)

4. Normalization analysis

First Normal Form (1NF): Attributes store single values rather than repeating lists. The items in an order are represented as separate rows in Order_Items, not as a comma-separated list in Orders. A customer's several addresses are separate rows in Addresses, not repeated columns in Customers.

Second Normal Form (2NF): The only composite primary key is (order_id, item_id) in Order_Items. Its non-key attributes, quantity and unit_price, describe that item on that particular order, so they depend on the whole key and not on order_id or item_id alone. All other tables have single-attribute primary keys, so partial dependencies cannot occur.

Third Normal Form (3NF): Customer, restaurant, owner, category, menu item, and delivery agent details are stored in their own relations. Non-key details are not copied into Orders, Payments, or Reviews. This removes transitive dependencies such as order_id → restaurant_id → owner_id, and order_id → customer_id → name. One point to note is Addresses: pincode may determine city, which would be a transitive dependency. It is kept in one table for simplicity because a pincode can cover more than one locality name; a separate Pincodes table could be added if strict 3NF is required.

5. Design improvements and integrity considerations

- Keep the current separation of master tables (customers, restaurants, menu items, agents), the junction table (Order_Items), and transaction tables (Orders, Payments, Reviews).
- Enforce UNIQUE on customer, owner and agent email or phone, vehicle number, category name, transaction_ref, (restaurant_id, name) for menu items, and on Payments.order_id and Reviews.order_id where these values are intended to identify unique records.
- Use CHECK constraints for status values (order_status, payment_status, opening_status), rating between 1 and 5, price and amount greater than or equal to zero, and quantity greater than zero.
- Add indexes to frequently searched foreign keys and fields such as Orders.customer_id, Orders.restaurant_id, Orders.agent_id, Menu_Items.restaurant_id and Orders.order_status when performance warrants them.
- Orders.total_amount and Payments.amount are derived from the sum of quantity × unit_price in Order_Items. This is controlled redundancy kept for speed and billing history; it should be set in the same transaction as the order lines so the values never disagree.
- Order_Items.unit_price intentionally copies the menu price at the time of ordering. Without it, a later change to Menu_Items.price would silently change the amount of past orders.
- An order should only contain menu items of the restaurant that received it, and its delivery address should belong to the customer who placed it. Plain foreign keys do not check this, so use composite foreign keys or a trigger/transaction to enforce it.

- Orders.agent_id must allow NULL until an agent is assigned. Add application logic to prevent an agent from being assigned to two overlapping deliveries and to keep Delivery_Agents.availability_status and Orders.order_status synchronized.
- Reviews should be accepted only for orders whose status is delivered. In Assignment 2, the relationships Orders → Payments and Orders → Reviews are described as one-to-one; the schema implements this with a UNIQUE order_id. If the platform later allows split or retried payments for one order, the Payments rule would need to change.
- If a customer edits or deletes a saved address, past orders would point to changed data. Storing a copy of the delivery address on the order, or marking addresses inactive instead of deleting them, would preserve delivery history.

6. Conclusion

The 11-table design reduces redundancy by storing each entity's facts in one place and using foreign keys for relationships. The remaining risks are mainly business-rule consistency across tables, such as matching restaurant and menu items, keeping totals and statuses in step, and agent assignment, which should be handled with constraints and transactional logic.